Algoritmi paraleli si distribuiți

Proiectarea programelor paralele /Tipuri de execuție paralelă

+OpenMP

Modele de programare paralelă

- Elementul cheie este sarcina (en., task).
- Un calcul paralel constă din una sau mai multe sarcini care se execută concurent.
- Un *task* este reprezentat de un program secvențial și memoria sa locală. De asemenea, acesta trebuie să aibă definită o intrefață (în sensul de API) de comunicare cu mediul. Înterfața este definită prin *inports* si *outports*, acestea specifică modul în care sunt transmise datele de intrare și cum se obțin rezultatele/raspunsurile.
- Un task poate exacuta următoarele 4 acțiuni în plus față de citire/scriere din/în memoria locală: trimiterea de mesaje la *outport,* recepționarea de mesaje prin *inports,* crearea unui nou task și terminarea taskului.

Modele de programare paralelă (cont.)

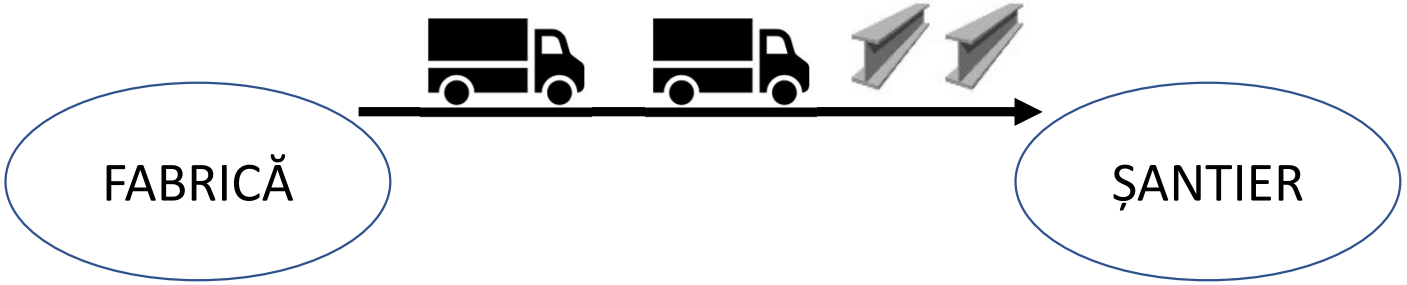
- Operația de trimitere este asincronă, aceasta se execută și se finalizează imediat.
- Operația de recepționare este sincronă, aceasta determină blocarea task-ului până când un mesaj este recepționat.
- Perechile outport/inport se pot conecta prin cozi de mesaje, denumite **canale**. Canalele pot fi create sau șterse.
- Task-urile pot fi asignate procesoarelor fizice; modul în care este asignat un task nu afectează semantica acestuia. Da, mai multe task-uri se pot aloca (en., *assign*) unui singur procesor.

Task-uri

- Utilizarea în mod abstract a task-urilor ne oferă un mecanism de înglobare a principiului localizării datelor:
  - Datele conținute în memoria locală a unui task sunt ”apropiate”; toate celelalte date sunt “îndepărtate”.
  - Utilizarea canalelor oferă un mecanism pentru a specifica faptul că ”un task are nevoie de date/rezultat din alt task pentru a putea continua”. Acest aspect poartă numele de dependința datelor (en., data dependency).
  - Întrebări:
    - Ce este un *computation graph?*
    - Ce algoritmi clasici din teoria grafurilor pot fi utili si în ce context?
    - Cum se se pot paraleliza astfel de algoritmi?

Task-uri: exemplu

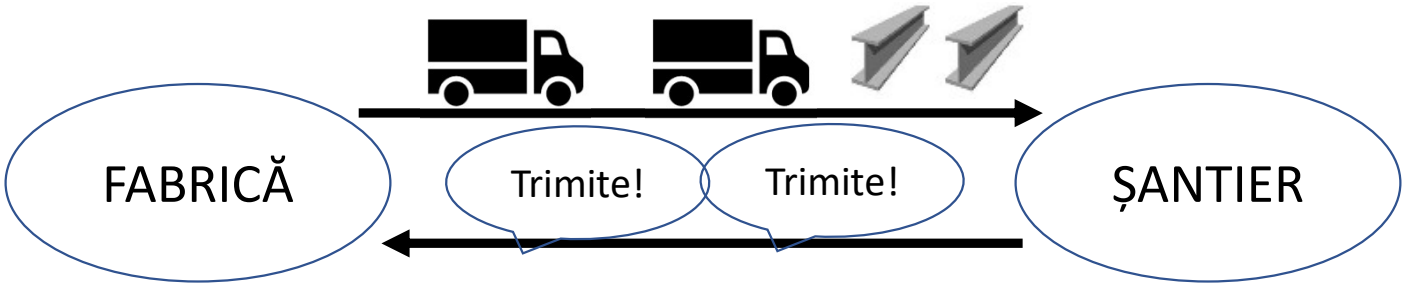
- Dorim construirea unui pod din traverse.
- Traversele sunt construite/încărcate la **FABRICĂ** și sunt asamblate pe **ȘANTIER**.
- Traversele sunt transportate cu camioanele de la fabrică la șantier.



- Fabrica si santierul pot reprezenta task-uri diferite.
- Limitare: Fapbrica poate produce traverse mai repede decât acestea pot fi montate pe șantier.

Task-uri: exemplu (cont.)

- Pentru a împiedica ȘANTIERUL să se umple de traverse trebuie ca inginerii să ceară explicit traverse atunci când rezervele scad.
- Această abordare îmbunătățită necesită un al doilea canal de comunicație care să informeze fabrica atunci când trebuie trimise traverse. Acest canal va anunta fabrica să oprească trimiterea de traverse atunci cand podul este finalizat.



- Abordare practică: mai multe drumuri, mai multe mașini, mai multe puncte de lucru pe șantier, etc.

Descompunerea și alocarea: Metodologia Foster

- Concepte și terminologie
  - Comunicare = schimbul de date între task-uri care se execută în paralel
    - Necesită timp
  - Sincronizare = coordonarea în timp real a task-urilor paralele
    - Conduce la așteptări și întârzieri
  - Granularitate
    - Mare => comunicare puțină, calcule/funcții mari
    - Mică => comunicare multă, calcule/funcții mici
  - Costurile suplimentare apar datorită
    - Sincronizărilor și schimburilor de date
    - Compilatorului, bibliotecilor, uneltelor, SO, etc.

Descompunerea

- Este primul pas în proiectarea/dezvoltarea aplicațiilor paralele
- Constă în identificarea task-urilor care pot fi executate concurent
  - Inițial, numărul de task-uri trebuie să fie cât mai mare (granularitate fină)
  - Granularitatea fină oferă cea mai mare flexibilitate în ceea ce privește proiectarea unui algoritm paralel.
  - Ulterior evaluarea cerințelor de comunicare, arhitectura sistemului pe care va rula aplicația sau aspectelele legate de proiectarea software vor definiexact modul de paralelizare
- Task-urile pot fi identice, diferite sau de dimensiune necunoscută la rulare
- Descompunerea se poate reprezenta printr-un ***graf de dependințe și interacțiuni ale task-urilor***.

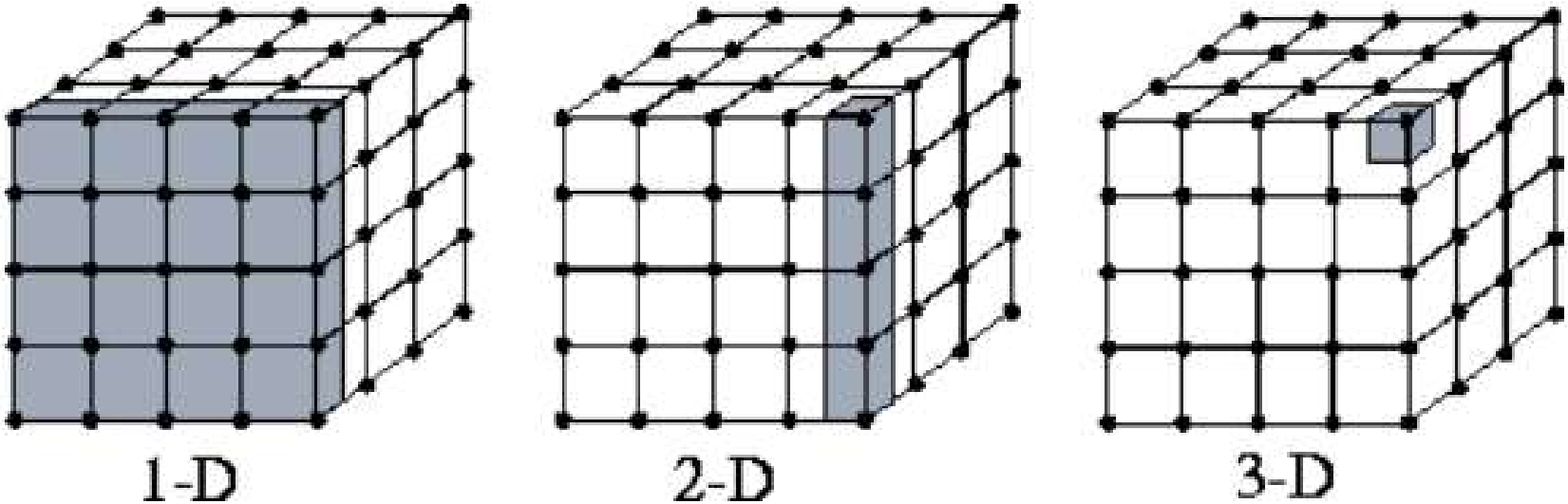
Descompunerea (cont.)

- O descompunere de bună calitate împarte atât **logica de calcul** necesară, cât și **datele** care trebuie prelucrate.
  - Primul pas se referă intotdeauna la descompunerea datelor.
  - Apoi, trebuie determinat modul în care sunt efectuate procesările pe datele descompuse
  - Obs: Acest mod de descompunere este denumit **descompunere de domeniu**.
- **Descompunerea funcțională** are în vedere mai întâi descompunerea procesării care trebuie efectuată, apoi are loc descompunerea datelor.
- Obs. Aceste două modalități de descompunere sunt complementare, dar pot fi aplicate la module diferile ale aceleiași probleme, sau pot fi aplicate la aceeași problemă pentru a obține algoritmi alternativi.

Descompunerea (cont.)

- Scopul principal al descompunerii este să evite replicarea procesărilor sau a datelor.
  - Descompunerea fină (en. *fine-grained*) oferă cea mai mare flexibilitate în ceea ce provește potențialul de paralelizare a algoritmului serial.
- Descompunerea domeniului (a datelor)
  - Ideal ar fi să putem împărți datele în părți egale
  - Apoi, împărțim calculele care trebuie efectuate prin alocarea fiecărei părți de calcul a datelor necesare pe care lucrează. Astfel se obțin task-uri
    - Task = datele si operațiile care trebuie efectuate pe acele date.
    - O operație poate necesita rezultate/date de la mai multe task-uri => nevoia de comunicare

Descompunerea domeniului



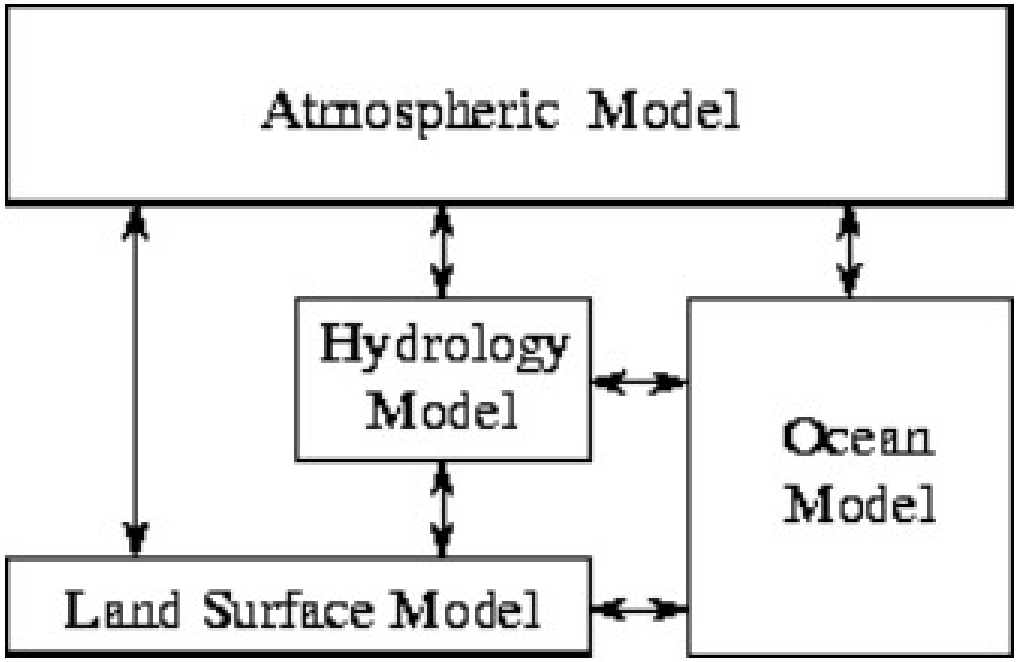
Descompunerea domeniului pentru o problemă cu date 3D. Datele care pot fi asociate cu un task sunt marcate cu gri. Descompunerea 3D este cea mai fină, oferă cea mai mare flexibilitate și este adoptată în stadiile inițiale ale proiectării

Descompunerea funcțională

- Reprezintă un mod alternativ de abordare a problemelor.
  - Atenția maximă este acordată procesărilor care trebuie efectuate, NU datelor.
  - Procesarea necesară trebuie împărțită în părți disjuncte; dacă datele necesare sunt de asemenea disjuncte atunci putem defi task-uri și descompunerea este finalizată.
  - Suprapunerile semnificative conduc la nevoia de comunicație în scopul evitării replicării datelor.
  - Trebuie avută în vedere în ceea ce privește investigarea diverselor modalități de paralelizare.

Descompunerea funcțională (cont.)

- Modelul de calcul al climei =
  - Modelul atmosferic determină viteza vântului, care este folosită de modelul oceanului
  - Modelul oceanului detrmină temperatura apei la suprafață, care este folosită de modelul atmosferic,
  - Etc.



Descompunerea funcțională (cont.)

- Ajută la modularizarea/structurarea programului/aplicației
- Modele complexe sunt de cele mai multe ori structurate/împărțite în colecții de modele simple care sunt conectate prin interfețe.
- Exemplu: Modelul climatic = colecție de modele (atmosfră, ocean, hidrologie, gheață, surse de dioxid de carbon, etc.)

Pașii necesari partiționării/descompunerii

- Partiționarea poate produce mai multe moduri de descompunere a problemei
- Pașii de verificare a corectitudinii sunt:

1. Cat de multe sunt task-urile în raport cu procesoarele pe care le avem disponibile pe masina care va executa programul?
   - Trebuie sa avem cel puțin un ordin de marime mai multe task-uri decat procesoare, altfel nu avem flexibilitatea necesară.
2. Descompunerea propusă evită procesările/nevoile de memorie redundante?
   - Dacă răspunsul este NU atunci propunerea nu va fi scalabilă pentru input mare.

Pașii necesari partiționării/descompunerii (cont.)

3. Sunt task-urile de dimensiune comparabilă?

- Dacă nu, adunci va fi dificil să alocăm fiecărui procesor o cantitate egală de calcule, așadar vom avea o problemă legată de balansarea încărcării.

4. Numărul de task-uri se scalează cu dimensiunea problemei?

- Ideal ar fi ca o creștere a dimensiunii problemei să determine o creștere a numărului de task-uri, si nu creșterea dimensiunii task-urilor existente. În această situație, algoritmul nu va rezolva probleme mai mari atunci când vom avea la dispoziție mai multe procesoare.

5. Ați identificat mai multe variante/opțiuni de descompunere?

- Maximizarea flexibilității în fazele avansate ale proiectări se poate asigura prin identificarea mai multor variante/opțiuni de descompunere, atât a domeniulu cât și funcțională.

Comunicarea

- Task-urile obținute prin descompunere sunt gândite să se execute concurent, dar de multe ori ele nu se pot executa independent.
- Comunicarea = fluxul de informații dintre task-uri.
- Poate fi specificată în 2 pași:
  - Definim un canal între task-un care cere date (ex., consumator) și task-ul care deține/calculează datele necesare (ex., producător).
  - Specificăm mesajele care vor fi trimise si primite prin canalele definite.
    - Obs. În funcție de tehnologie/implementare codarea algoritmului poate să nu necesite crearea explicită a canalelor. Totuși, înțelegerea acestor aspecte permite evaluarea calitativa și cantitativă a problemelor legate de costurile de comunicare și de localizarea datelor.

Comunicarea: tipuri

- Locală/globală:
  - Locală: task-ul comunică cu o mulțime mică de alte task-uri ”vecine”
  - Globală: fiecare task comunică cu multe alte task-uri
- Structutată/nestructurată
  - Structurată: se folosește o structură regulată, ex. Arbore sau grid.
  - Nestructurată: rețeaua poate fi un graf oarecare.
- Static/dinamic
  - Static: identitatea task-urilor cu care se comunică NU se schimbă în timpul rulării
  - Dinamic: task-urile cu care se comunică sunt determnate la rulare și se pot schimba în timp.
- Sincron/asincron
  - Sincron: producătorii și consumatorii se execută coordonat
  - Asincron: consumatorul obține date fără a coopera cu producătorul

Comunicare locală: exemplu

$$X[i,j]^{(t+1)} = (4 * X[i,j]^{(t)} + X[i-1,j]^{(t)} + X[i+1,j]^{(t)} + X[i,j-1]^{(t)} + X[i,j+1]^{(t)})/8$$

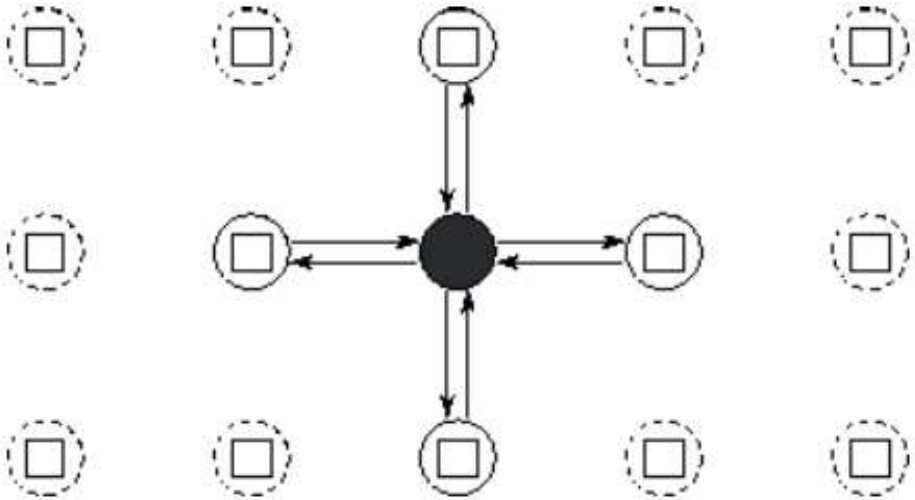
Task-urile si structura de canale pentru calculul diferenței finite într-un tabel 2D cu un șablon de updatare în 5 puncte (folosit ca metodă numerică simplă pentru metoda Jacobi de calcul a diferențelor finite).

- Descompunerea domeniului:

Un task alocat punctului X[i,j] trebuie să determine secvența:

$X[i,j]^{(1)}$, $X[i,j]^{(2)}$, $X[i,j]^{(3)}$, ...

... Dar fiecrare calcul are nevoie de 4 secvențe corespunzătoare punctelor vecine.



Comunicare locală: exemplu (cont)

- Fiecare task va trebui să execute următorul algoritm

for t=0 to T-1

send $X[i,j]^{(t)}$ *către fiecare vecin*

primește $X[i-1,j]^{(t)}$, $X[i+1,j]^{(t)}$, $X[i,j-1]^{(t)}$, $X[i,j+1]^{(t)}$ *de la vecini*

determină $X[i,j]^{(t+1)}$ *conform cu ecuația*

- Toate punctele pot fi updatate concurent!

Comunicarea globală

- Avem multe task-uri care participă.
- Nu este suficient să identificăm perechile producător/consumator.
- O astfel de abordare ar implica prea multe canale de comunicare sau ar putea restricționa oportunitățile de execuție concurentă.
- Problema sumei centralizate

$$\mathbf{S = sum_{i=0,\ N-1}\ X_i}$$

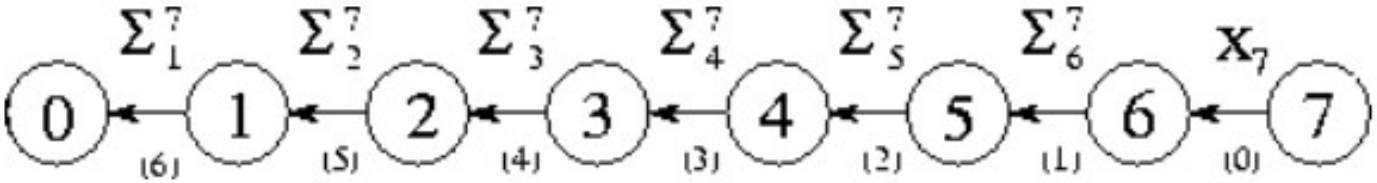
- Presupunem ca un task cere efectuarea acestui calcul, prin cererea valorilor X_0, X_1, ... de la task-urile 0, 1, ... Asadar am putea proiecta o structură de comunucare care să permită comunicarea fiecărui task cu task-ul S (cel care doreste să efectueze suma) în mod independent. Totusi, deoarece S poate primi și aduna o singură valoare la un moment dat, înseamnă că avem nevoie de O(N) timp, ceea ce nu reprezintă o paralelizare bună!

Comunicarea globală (cont.)

- Aspecte care opresc/împiedică paralelismul
  - Algoritmul este ***centralizat***: el nu distribuie nici calculele și nici comunicarea. Un singur task (i.e., managerul S) trebuie să participe la fiecare operație/calcul.
  - Algoritmul este ***secvențial***: Nu permite ca efectuarea concurentă de calcule sau comunicare.
- Soluție:
  - Distribuirea comunicării și calculelor!

Distribuirea comunicării și calculelor

- Distribuim calcularea sumei celor N numere alocând fiecărui task *i* sarcina de a detrmina suma **S[i] = X[i] + S [i-1]**



- Task-ul N-1 trimite valoarea calculată vecinului său
- Se distribuie N-1 adunări și transmiteri, dar execuția concurentă este posibilă numai dacă mai multe adunări se execută concurent.
- Determinarea sumei costă încă N-1 pași.
- Soluția: Divide and Conquer

Utilizarea *divide and conquer* pentru paralelizare

- Împărțim problema în două sau mai multe probleme mai simple de dimensiune aproximativ egală (ex., adunpm N/2 numere).
- Acest proces este aplicat recursiv până când problema nu mai poate fi împărțită (i.e., avem de adunat două numere).
- Strategia este eficientă d.p.d.v. Al paralelizării atunci când subproblemele pot fi rezolvate concurent.
- Pentru $N = 2^n$, $n > 0$ avem:

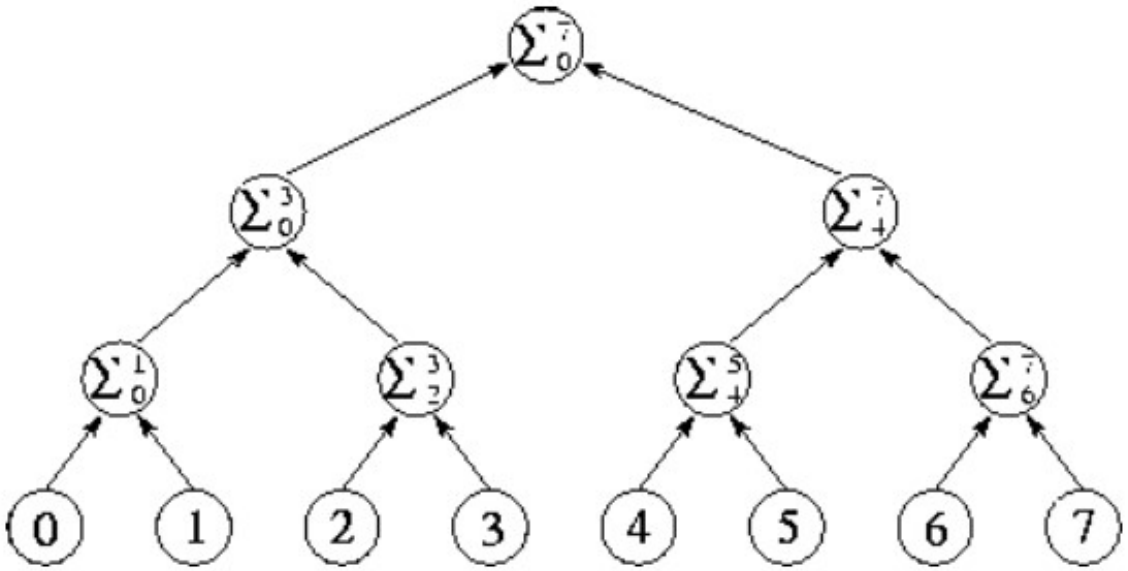
$$\mathbf{Suma}_{i=0,\, 2^{n-1}} = \mathbf{Suma}_{i=0,\, 2^{n-1}-1} + \mathbf{Suma}_{i=2^{n-1},\, 2^{n}-1}$$

- Cele două sume pot fi executate concurent.

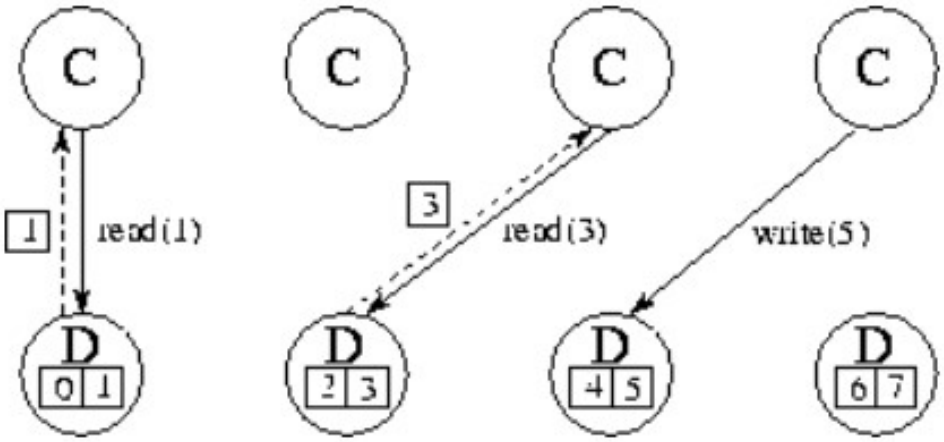
Utilizarea *divide and conquer* pentru paralelizare (cont)

- Sumele care se găsesc pe același nivel pot fi executate concurent.
- Arborele are înlțime

  h = log N
- Detrminarea sumei se realizează în ***log N*** pași în loc de ***N***.
- Structura de comunicare este
  - regulată, fiecare task comunică cu un număr mic de vecini.
  - Sincronă: task-urile producător trimit explicit rezultatele către consumator.



Comunicarea asincronă



- Avem task-uri *de date (D)* separate pentru efectuarea cererilor de citire și scriere pe o structură de date distribuită.

- Task-urile *de calcul (C)* trimit cereri de citire/scriere către cele 8 celule/elemente de date distribuite la cele 4 task-uri D.
- Liniile pline reprezintă cereri; liniile întrerupte reprezintă răspunsuri.
- Un task C și un task D pot fi plasate pe fieceare dintre cele 4 procesoare pentru a distribui datele și procesările echitabil.

Comunicarea asincronă (cont.)

- Scenariul prezentat anterior apare atunci când procesarea este structurată ca o mulțime de task-uri care priodic efectuază operații de citire/scriere din/în elementele unei structuri de date partajată.
  - Presupunem ca această strucură de date este prea mare sau este prea frecvent accesată pentru a putea fi încapsulată într-un singur task.
  - Așadar, este nevoie de un mecanism de distribuire a structurii de date care să perimită operații asincrone de citire sau scriere asupra elementelor ei.
- Mecanisme posibile
  - Structura de date este distribuită între task-uri C (de calcul). Fiecare task efectuază atât procesări cât și cereri de date localizate la alete task-uri. Task-ul trebuie sa-și întrerupă periodic execuția proprie și să răspună cererilor care sunt în așteptare.

Comunicarea asincronă (cont.)

- Mecanisme posibile (cont.)
  - Structura de date distribuită este încapsulată într-o mulțime separată de task-uri care raspund numai la cererile de citire sau scriere.
  - Pe sistemele acre permit modelul de programare cu memorie partajată, task-urile C (de calcul) pot accesa datele partajate fără a fi nevoie de alte mecanisme. Totuși, trebuie verificată corectitudinea ordinii în care se efectuază operațiile deoarece putem ajunge în situație de nedeterminism.
- Caracteristicile de performanță variază de la un sistem la altul.
- Limitări
  - Prima strategie poate conduce la programe nemodulare. Pooling-ul (răspunsul la cereri în așteptare) este o operație costisitoare pe unele sisteme.
  - Cea de-a doua strategie nu nu exploatează localizarea datelor. Practic, nu există date locale.

Verificarea modului de comunicare

Evaluarea modului în care a fost proiectată comunicarea

1) Toate task-urile definite realizează aproximativ același număr de oprații de comunicare?

- Operațiile de comunicare trebuie distribute cat mai uniform.

2) Fiecare task comunică cu un număr mic de vecini?

- Daca toate task-urile comunică cu multe alte task-uri trebuie luată în calcul abordarea cu comunicare locală în locul comunicării globale.

3) Operațiile de comunicare se desfășoară în paralel?

Dacă nu, atunci algoritmul nu este scalabil. Obținerea concurenței poate fi făcută prin *divide and conquer*.

3) Procesările/calculele asociate task-urilor se desfășoară concurent ?

Dacă nu, algoritmul nu este scalabil și trebuie avută în vedere reordonarea operațiilr de comunicare și procesare.

Ghid de paralelizare (Foster)

- Identifică modulele cu necesități mari de calcul (en., computational hotspots)
  - Determină ceea ce merită să fie paralelizat
- Partiționează/Împarte problema în task-uri mici și cât mai independente
  - Determină/crează/definește paralelismul
- Identifică necesitățile de comunicare dintre task-uri
  - Stabilește constrângerile de comunicare dintre task-uri
- Aglomerează task-urile mai mici în task-uri mai mari
  - Grupează task-urile de bază impreună astfel încât comunicarea să devină minimă. Mare atenție la balansarea încărcării.
- Alocă task-urile și datele pe procesoare
  - Balansează încărcarea procesoarelor încercând sa minimizezi comunicarea

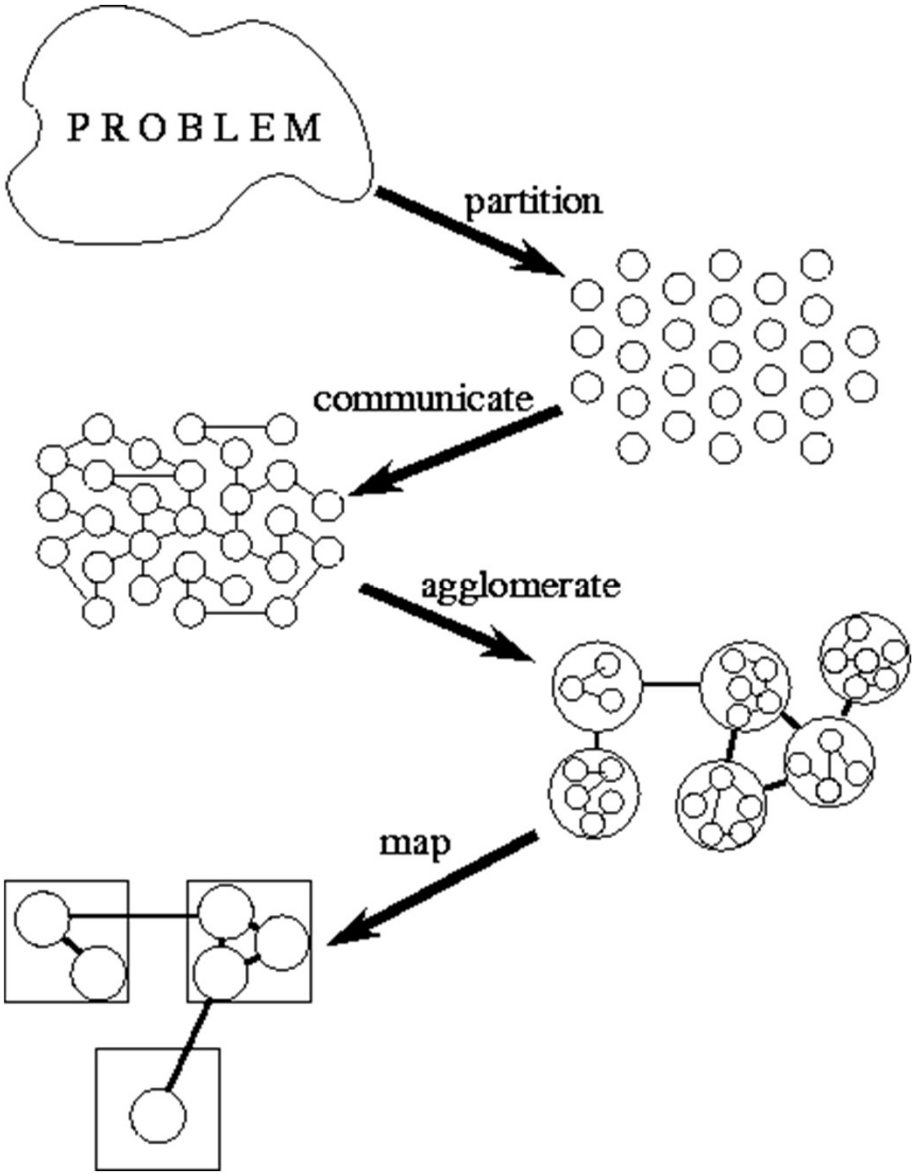
De la descompunere la alocare

- Metodologia lui Foster exemplifică procesul de dezvoltare software care este strâns legat de proiectarea paralelă:
- Descompunem problema în task-uri care au potențial să fie executate în paralel
  - Descompunerea poate fi reprezentată printr-un graf de dependințe și interacțiuni între task-uri
  - O problemă poate fi descompusă în multe moduri în task-uri; acestea pot fi de aceeași mărime sau de marimi diferite.
- Analizăm nevoile de comunicare între task-uri
- Grupăm task-urile care comunică intens
- Asignăm task-urile pe procesoare

Criteriile de alocare

- Trebuie maximizată concurența prin alocarea task-urilor independente la procesoare diferite
- Timpul total de execuție trebuie minimizat prin prioritizarea task-urilor care se află pe calea critică a grafului de dependințe.
- Minimizarea interacțiunii trebuie relizată prin alocarea task-urilor cu grad mare de interacțiune mutuală la același proces.

Proiectarea algoritmilor paraleli

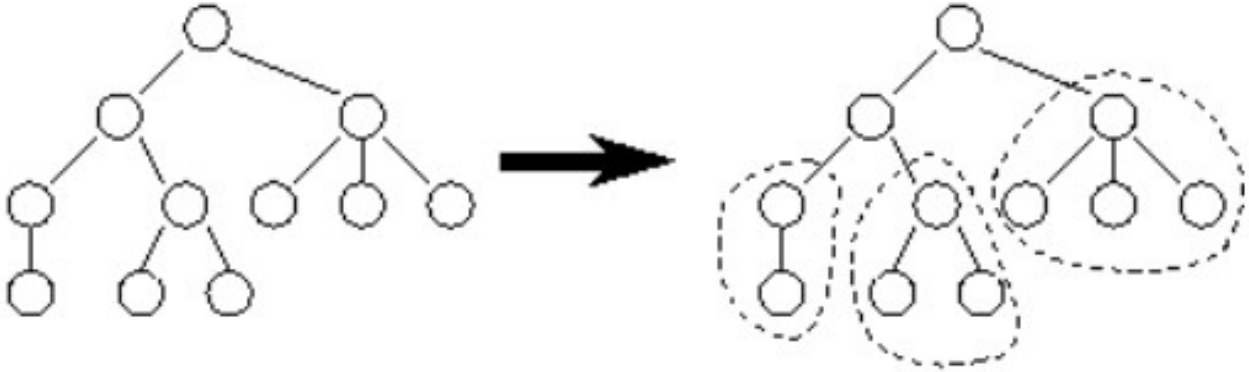


Proiectarea algoritmilor paraleli

- **Scop**: Pentru o anumită problemă determinați un algoritm care îndeplinește condițiile de concurență, scalabilitate și localizare.
  - Concurență: avem prelucrări paralele
  - Scalabilitate: Algortmul scalează bine (liniar) pentru date de intrare mai mari, sistem cu mai multe procesoare, memorie mai multa, etc.
  - Localizare: alocarea task-urilor si datelor este eficientă.
- Cele mai multe probleme
  - Au mai multe soluții algoritmice seriale; totuși, presupunem ca avem la dispoziție soluția optimă.
  - Au și mai multe soluții paralele. Ingredientele unei soluții paralele sunt: gradul de descompunere, graful de dependințe al task-urilor, modul de alocare al task-urilor, implementarea.

Aglomerarea

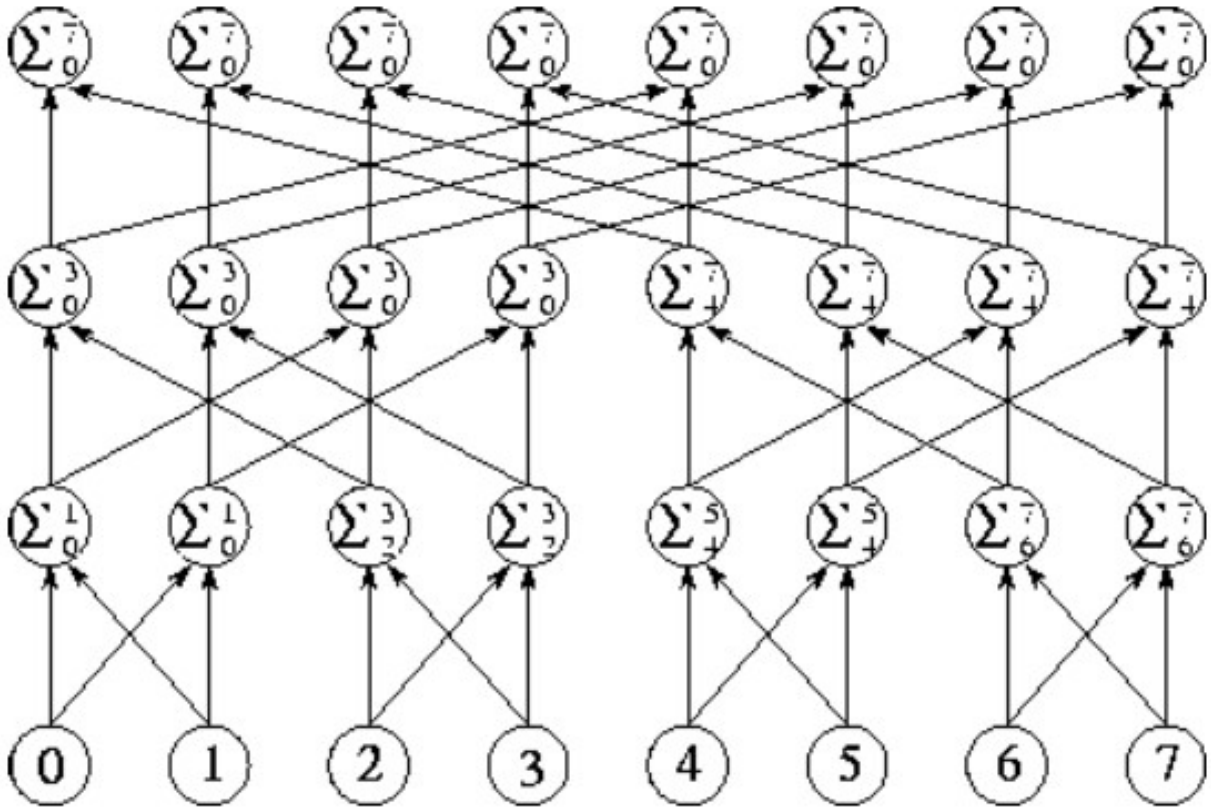
- Are ca scop definirea unui număr mai mic de task-uri de dimensiune mai mare



- Subarborii obținuți prin *divide and conquer* determină task-uri mai mari.
  - Proiectarea este abstractă deoarece problema alocării task-urilor la procesoare rămâne nerezolvată
  - Soluție: definim cate un task pentru fiecare procesor.
- Scopurile aglomerării: (1) reducerea comunicării; (2) păstrarea flexibilității în ceea ce privește scalabilitatea și alocarea task-urilor; (3) reducerea costurilor de dezvoltare software.

Aglomerarea: Structura fluture (en., butterfly)

- log N nivele;
- fiecare task primește date de la două task-uri, efectuează o singură operație de adunare și transmite rezultatul către doup taskuri aflate la nivelul superior
- Suma celor N valori este efectuată în **log N** pași.
- Suma completă este determinată de fiecare task al nivelului superior.



Aglomerarea: Verificarea corectitudinii proiectării

- S-au redus costurile de comunicare prin creșterea localizării?
  - Dacă nu, trebuie adaptată strategia de aglomerare.
- Au apărut procesări replicate?
  - Dacă da, beneficiile replicarii sunt mai mari decât costurile? Care este influența dimensiunii problemei și a numărului de procesoare?
- Trebuiesc replicate date?
  - Dacă da, este compromisă scalabilitatea algoritmului prin limitarea dimensiunii problemei sau a numărului de procesoare?
- Am obținut task-uri cu costuri similare de procesare și comunicare?
  - Cu cât sunt mai mari task-urile definite cu atat trebuie sa fie mai similare.
  - Dacă avem un task per procesor atunci acestea trebuie să aibă costuri aproximativ identice.

Aglomerarea: Verificarea corectitudinii proiectării (cont.)

- Numărul de task-uri scalează cu dimensiunea problemei?
  - Dacă nu, atunci algoritmul nu va putea rezolva probleme mai mari pe sisteme cu mai multe procesoare.
- Au fost eliminate posibilități de execuție concurentă?
  - Un algoritm cu concurență insuficientă poate deveni cel mai eficient dacă ceilalți algoritmi au costuri excesiv de mari de comunicare.
  - Alegerea se face prin evaluarea performanțelor.
- Mai poate fi redus numărul de task-uri fără a produce dezechilibrarea încărcării, creșterea costurilor de dezvoltare sau reducerea scalabilității?
- Care sunt costurile de modificare a versiunii secvențiale pentru obținerea paralelismului?

Alocarea

- Cel de-al 4-lea si ultimul pas al proiectării
- Specifică unde se execută fiecare task
- Nu nu are sens în sistemele cu un singur procesor sau pe sistemele cu memorie partajată. Acestea au mecanisme automate de planificarea (en., schedule).
- Este o problemă dificilă care trebuie adresată explicit în procesul de proiectare al algoritmilor paraleli.
- Are ca scop minimizarea timpului total de execuție.

Alocarea (cont.)

- Există două strategii pentru minimizarea timpului total de execuție
  - Alocăm task-urile care se pot executa concurent pe procesoare *diferite* .
    - scopul este concurența crescută.
  - Alocăm task-urile care comunică frecvent pe *același* procesor
    - scopul este localizarea crescută.
- Aceste strategii pot intra în conflict!
  - Si atunci este nevoie de compromis.
  - Mai mult, resursele limitate pot diminua numărul de task-uri care se pot aloca unui singur procesor.

Alocarea (cont.)

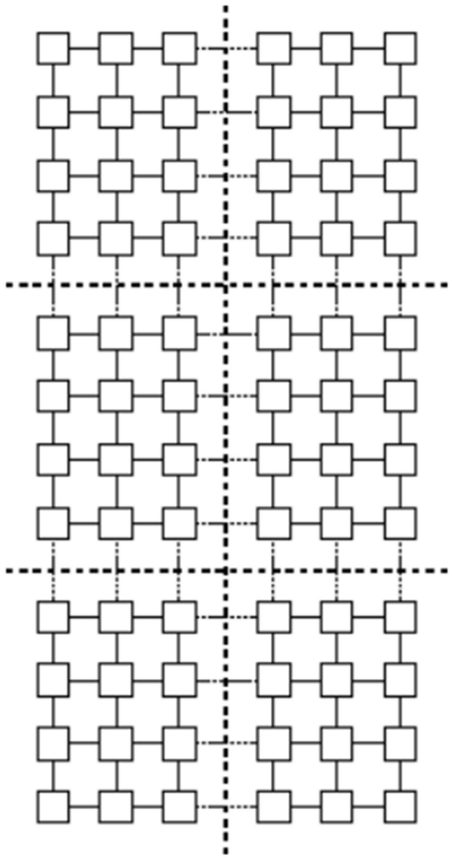
- Problema alocarii este *NP-complete*, adică nu există un algoritm polinomial (i.e., en. computationally tractable) care să evalueze compromisurile pe caz general.
- Totuși:
  - Există strategii specializate, euristici și clase de probleme care conduc la performanță.

Alocarea

Context: număr fix de task-uri; task-uri de dimensiune egală; comunicare structurată locală și globală.

Alocarea eficientă este directă: alocăm task-urile astfel încât să minimizăm comunicarea între procese.

- Avem 6 procesoare
- Se alocă fiecărui procesor aceeași cantitate de procesat
- Se minimizează comunicarea între procesoare.



Alocarea

- Dacă avem dimensiuni variabile/diferite ale nevoilor de procesare, și/sau comunicare nestructurată atunci alocarea poate să nu fie evidentă.
  - În această situație este nevoie de algoritmi pentru balansarea încărcării care să determine alocarea eficientă. De obicei, determinarea este euristică.
  - Timpul necesar rulării acestor algoritmi trebuie luat în calcul.
  - *Algoritmii probabilistici de balansare* aduc de multe ori costuri suplimentare mai mici decât cei care se bazează e structura inițială a soluției.
  - *Algoritmii dinamici de balansare* sunt utili atunci când numarul de task-uri, dimensiunea procesării necesare sau cenesitățile de comunicare se schimbă dinamic în timpul execuției.
  - *Algoritmii de programare a task-urilor* (en. Task scheduling) sunt folositi în situațiile descompunerii funcționale, când multe task-uri mici necesită coordonare la începutul și la sfârșitul execuției.

Alocarea: Algoritmi de echilibrare a încărcării

- Mai sunt cunoscuți ca algoritmi de *partiționare*.
- Tehnica bisecțiunii recursive (en. recursive bisection)
  - Partiționează un domeniu (ex., un grid finit) în subdomenii de cost aproximativ egal, încercând să minimizeze costurile de comunicare (i.e., canalele de comunicare care traversează granițele).
  - Abordarea clasică este *divide and conquer*
    - O primă partiționare divide domeniul în două subdomenii
    - Partiționarea recursivă continuă până când numărul de subdomenii este egal cu nbumărul de task-uri dorit.
  - Tehnica *bisecțiunii recursive coordonate* se aplică gridurilor neregulate
  - Tehnica *bisecțiunii recursive de tip graf* se aplică grid-urilor cu structură graf.
    - Se identifică două extremități ale grafului
    - Fiecare nod este alocat extremității apropiate

Alocarea: Algoritmi de echilibrare a încărcării (cont.)

- Algoritmi locali
  - Tehnicile anterioare necesită conoașterea stării globale a stării în care se află procesarea
  - Algoritmii locali de echilibrare folosesc numai informațiile obținute de un număr mic de procesoare vecine (i.e., abordare *greedy*).
  - Ex. Dacă procesoarele sunt organizate intr-un *mesh* (i.e., plasă) acestea compară starea de încărcare cu cea a vecinilor și decid să tranfere execuția de task-uri dacă diferența dintre încărcări depășește un anumit prag (en., threshold).
    - Cum putem detrmina pragul optim? Experimental!

Alocarea: Algoritmi de echilibrare a încărcării (cont.)

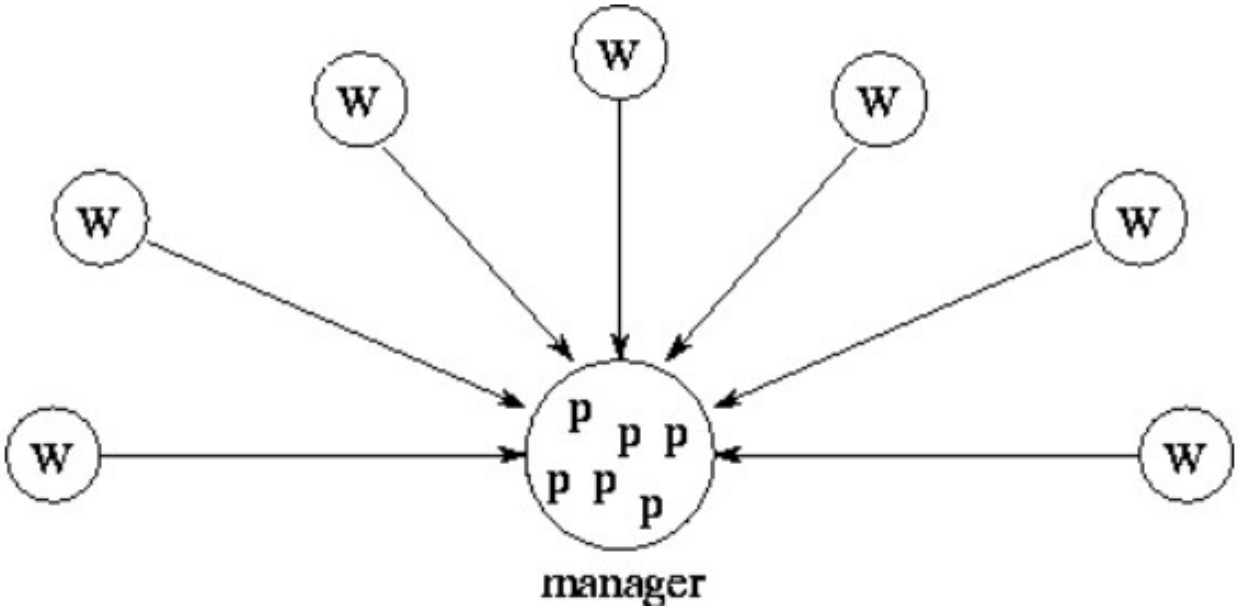
- Algoritmi probabilistici
  - Abordare simplă care alocă task-urile în mod aleator procesoarelor.
  - Dacă numarul de task-uri este mare atunci fiecare procesor va avea alocată aproximativ aceeași catitate de procesare.
  - Avantaje: cost mic și scalabilitate.
  - Dezavantaje: este nevoie de comunicare, iar echilibrarea devine acceptabilă dacă numărul de task-uri este mult mai mare decât numarul de procesoare.
  - Strategia este bună atunci când avem nevoie de puțină comunicare între task-uri.

Alocarea: Algoritmi de programare a task-urilor (en., scheduling)

- Sunt utili atunci când descompunerea funcțională produce multe task-uri, fiecare dintre aceste având cerințe mici/slabe de localizare.
- O mulțime (en., pool) distribuită/centralizată de task-uri este creată și administrată.
  - Aici este adăugat un task nou.
  - De aici este luat un task pentru a fi alocat unui procesor.
  - Task-urile devin structuri de date reprezentând ”problemele” care trebuie rezolvate de o mulțime de procesoare.
- Strategia de alocare/programare reprezintă un compromis între cerințe conflictuale ale task-urilor (acestea doresc rularea independentă pentru a reduce costurile de comunicare) și starea generală a procesării (necesară balansării).

Alocarea: Algoritmi de programare a task-urilor

- Structura manager/worker
- **Worker** = face cereri repetate și rezolvă probleme alocate de manager.
  - Poate trimite managerului task-uri noi! ... Iar managerul le va aloca altor worker-i.
- **Manager** = gestionează o mulțime de probleme (p) și răspunde cererilor venite de la worker-i.



Alocarea: Algoritmi de programare a task-urilor

- Structura ierarhizată manager/worker
  - Împarte worker-i în mulțimi disjuncte, fiecare cu un sub-manager
  - Worker-ii cer task-uri de la sub-manageri, care comunică regulat cu managerul și cu alți sub-manageri pentru a balansa încărcarea procesoarelor de care răspund.
- Structuri descentralizate
  - Nu există manager. Fiecare procesor administrează o mulțime separată de task-uri.
  - Practic, mulțimea de task-uri devine o structură de date distribuită care poate fi accesată de task-uri diferite în mod asincron.
  - Politici de acces
    - Un worker poate face o cerere la un număr limitat predefinit de ”vecini”

Alocarea: Verificarea corectitudinii proiectării

- Ați luat în considerare un algoritm de alocare bazat pe crearea și distrugerea dinamică a task-urilor?
  - Algoritmul poate fi simplu dar demonstrarea performanței este dificilă.
- În situația în care utilizați o metodă de balansare centralizată, ati verificat ca managerul nu reprezintă el însuși o gâtuire (en., bottleneck)?
  - Puteți reduce costurile de comunicare prin transmiterea de pointeri către task-uri, în loc să transmiteți task-ul însuși (sau toate datele) către manager.
- În situația în care utilizați o metodă de balansare dinamică, ati evaluat mai multe strategii/variante?
  - Variantele probabilistice sunt simple si trebuie avute în vedere.
- În situația în care utilizați o metodă de balansare probabilistică, aveti un număr suficient de mare de task-uri pentru a asigura o balansare adecvată?
  - De obicei, trebuie sa aveți de 10 ori mai multe task-uri decât procesoare.

Concluzii

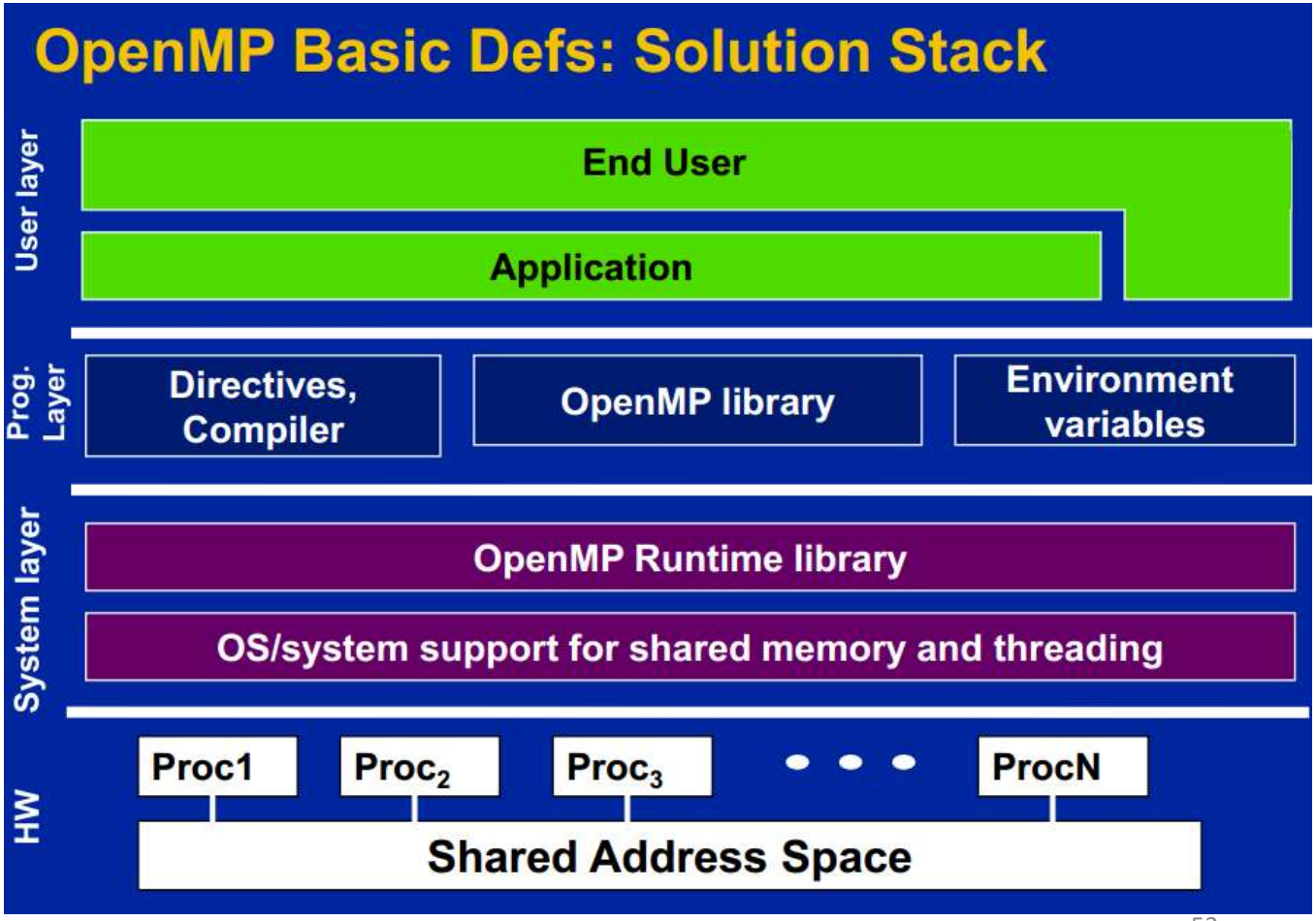
- Descompunerea
  - NU depinde de arhitectura sistemului
  - Trebuie să permită concurența.
- Aglomerarea
  - NU depinde de arhitectura sistemului.
  - Trebuie să echilibreze încărcarea și să limiteze comunicarea.
- Alocarea
  - Depinde de arhitectura sistemului.
  - Grupează procesele care au legătură între ele pe procesoare.
  - Exploatează localizarea în situația unei rețele.
- Orchestrarea generală a aplicației [NU depinde de arhitectura sistemului]
  - Reduce comunicarea prin localizare
  - Reduce serializarea rezurselor partajate
  - Planifică task-urile pentru a satisface dependințele cât mai din timp.

Concluzii (cont.)

- Orchestrarea generală a aplicației
  - Memoria partajată
    - Datele private și partajate sunt explicit declarate/separate
    - Sincronizarea se realizează prin operații atomice asupra datelor partajate
    - Sincronizarea este explicită și distincă față de comunicare
  - Transmiterea mesajelor
    - Are nevoie de distribuirea datelor între memoriile locale
    - Nu există date explicit partajate
    - Comunicarea este explicită
    - Sincronizarea este asigurată implicit de comunicare.

Open MP

- Este o mulțime de directive compilator și funcții de biblotecă pentru programarea paralelă.
- Simplifică dezvoltarea aplicațiilor cu fire de execuție îm C/C++/Fortran



Sintaxa OpenMP

- Cele mai multe constructe în OpenMP sunt directive compilator

```
#pragma omp construct [clause [clause]…]
```

```
#include<omp.h> //OpenMP include file
void main(){
  // parallel region with three threads
  #pragma omp parallel num_threads(3)
   {// Start al codului paralel

// Runtime library function to return a thread ID
int tid = omp_get_thread_num();
   printf("Hello world from thread = %d \n",tid);
   if(tid == 0){
      // Runtime library function to return the number of threads
      int nthreads = omp_get_num_threads();
      printf("Number of threads = %d\n",nthreads);
   }
  }// End al codului paralel
 }
```

Folosirea directivelor

In Visual Studio activați suportul OpenMP:
https://msdn.microsoft.com/de-de/library/fw509c3b(v=vs.120).aspx

In *gcc* trebuie să aveti -*fopenmp* pentru linking și compilare

Cum interacționează firele în OpenMP?

- OpenMP folosește un model multi-thread cu memorie partajată
  - Firele comunică prin partajarea variabilelor
- Partajarea neintenționată/întâmplatoare a datelor/variabilelor determină apariția situațiilor de *data race* sau *race conditions*
  - *data race* = această situație apare atuci când programul are raspunsuri diferite în funcție de ordine de planificare/executare a firelor.
- Controlul zonelor de *data race* se face prin sincronizare.
- Sincronizarea este costisitoare, așadar:
  - Modul de acces la date trebuie făcut astfel încât nevoia de sincrinizare să fie minimă.

Modelul OpenMP

- Paralelismul Fork-Join
  - Firul ***Master*** creează fire de excuție la nevoie.
  - Paralelismul este adăugat incremental până când se obțin performanțele necesare
    - Programul secvențial se transformă în program paralel
- Crearea firelor

```
omp_set_num_threads(4); // funcție care cere crearea unui anumit număr de fire.
```

- Declararea regiunilor paralele

```
#pragma omp parallel num_threads(4)
{      // instrucțiune care declară o regiune paralelă care cere crearea de 4 fire
}     // fiecare fir execută o copie a codului din blocul declarat
```

Modelul OpenMP

```
double A[1000];
#pragma omp parallel num_threads(4)
{
  int ID = omp_get_thread_num();
  myFunction(ID, A);
}
printf("Gata!\n");
```

O singură copie vectorului A este partajată de toate firele!

Comportamentul este similar cu

```
pthread_t tid[4];
for (int i = 1; i < 4; ++i)
        pthread_create (
                tid[i],0,thunk, 0);
ToDo();// the logic
for (int i = 1; i < 4; ++i)
        pthread_join (tid[i]);
```

Sincronizarea in OpenMP

Sincronizarea este folosită pentru a impune constângeri referitoare la ordinea de acces și pentru a proteja accesul la datele partajate.

- **Bariere**: Fiecare fir așteaptă la barieră până când ajung toate firele.
- **Excludere mutuală**: Definim o bucată de cod care poate fi executată de un singur fir la un moment dat.
- Sincronizare de nivel înalt
  - ***Critical***
  - ***Atomic***
  - ***Barrier***
  - Advanced: order, flush și locks.

Sincronizarea cu *barrier*

Barrier: fiecare fir așteaptă până când toate firele ajung la barieră.

```
#pragma omp parallel
{
int id = omp_get_thread_num();
A[id] = doSomething(id);
#pragma omp barrier
B[id] = doSomethingElse(id, A);
}
```

Sincronizarea cu *critical*

- **Excluderea mutuală**: Numai un fir poate intra în secțiunea critică.

```
float res;
#pragma omp parallel
{       float B; int i, id, nthrds;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads();
        for(i=id;i<niters;i+=nthrds){
                B = big_job(i);
#pragma omp critical    ;// firele își așteaptă rândul; numai un fir poate apela small_job()
                res += small_job (B);
        }//end for
}
```

Sincronizarea cu *atomic*

- Atomic oferă/implementează excludere mutuală dar se aplică numai la updatarea unei locații de memorie.

```
#pragma omp parallel
{
        double tmp, B;
        B = DOIT();
        tmp = big_ugly(B);
#pragma omp atomic
        X += tmp;
}
```

Alte constructe

- **Loop worksharing construct** împarte iterațiile din buclă între fire.

```
#pragma omp parallel
{
#pragma omp for
        for (I=0;I<N;I++){
                COOL_STUFF(I);// Variabila I este implicit ”privată” pentru
                                // fiecare fir.
        }//end for
}
```

Alte constructe (cont)

- Intrucțiunea **schedule** specifică modul în care iterațiiole din buclă sunt alocate la fire:
  - schedule(static [,chunk]);// Distribuie fiecărui blocuri de idetații de dimensiune “chunk”.
  - schedule(dynamic[,chunk]);// Fiecare fir preia “chunk” iterații din coadă.
  - schedule(guided[,chunk]);//Firele preiau dinamic blocuri de iterarții.
  - schedule(runtime);// Planificarea și dimensiunea chunk-ului este luată din variabila de mediue OMP_SCHEDULE.
  - schedule(auto);//Planificarea este la dispoziția mediului.

Task-uri în OpenMP

- Task-urile sunt alcătuite din
  - Codul care se execută
  - Datele din medie (en., data environment)
  - Variabile de control intern (en., internal control variables:ICV)
- Firele execută fiecrae task.
- Sistemul (i.e., runtime system) decide când se execută task-urile
  - Task-urile pot fi amânate sau executate imediat

Referințe

- https://www.openmp.org/resources/tutorials-articles/
- https://www.openmp.org/wp-content/uploads/Intro_To_OpenMP_Mattson.pdf